Introduction to Programming

W13 Concurrency

Stephan Krusche

21 January 2026
Technical University of Munich

Schedule

#	Date	Subject
1	15.10.25	Introduction
2	22.10.25	Control Flow in Programming
3	29.10.25	**In-Depth Core Concepts***
4	05.11.25	Core Data Structures
5	12.11.25	Code Reuse and Structure
6	19.11.25	Type Flexibility and Safety
7	26.11.25	**In-Depth Object Orientation***
8	03.12.25	Functional Programming Essentials
9	10.12.25	Algorithms and Data Handling
10	17.12.25	Programming Languages
11	07.01.26	Graphical User Interfaces
12	14.01.26	Recursion
13	**21.01.26**	**Concurrency**
14	28.01.26	Beyond Programming
15	04.02.26	Course Review

*** Repetition**

Roadmap

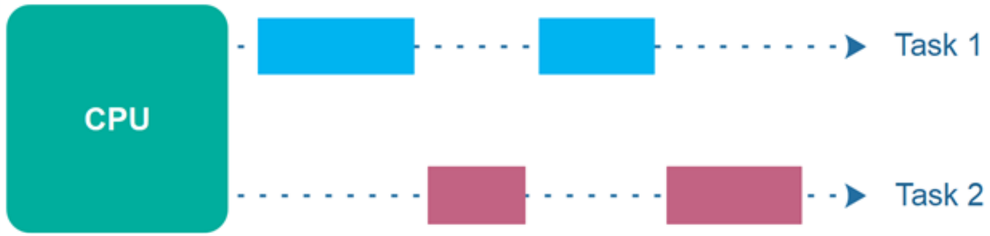
- **Context**
  - Apply OOP concepts: abstraction, encapsulation, inheritance and polymorphism
  - Use control structures and data types
  - Use enums, annotations, generics, lambda expressions, and streams
- **Learning goals**
  - Understand the need for concurrency in modern applications
  - Understand asynchronous programming with futures
  - Use completable futures for more control and composition
  - Understand when and how to use parallel streams effectively for efficient data processing
  - Understand the principles of reactive programming: responsiveness, resilience, elasticity, and message-driven architecture
  - Apply the publisher-subscriber model

Outline

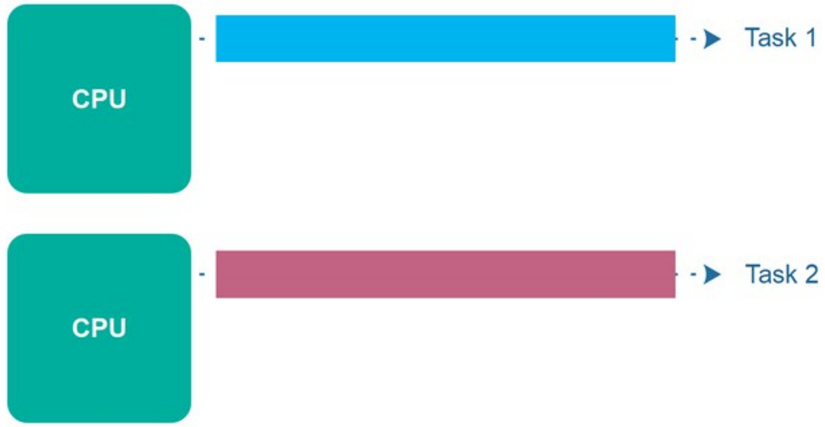
- **Modern concurrency**
- Task-based concurrency
- Completable futures
- Parallel streams
- Reactive programming

Why concurrency matters

- Keep applications responsive (e.g., user interfaces, web servers)
- Improve performance by leveraging multiple processor cores
- Essential for handling asynchronous events (e.g., network requests)

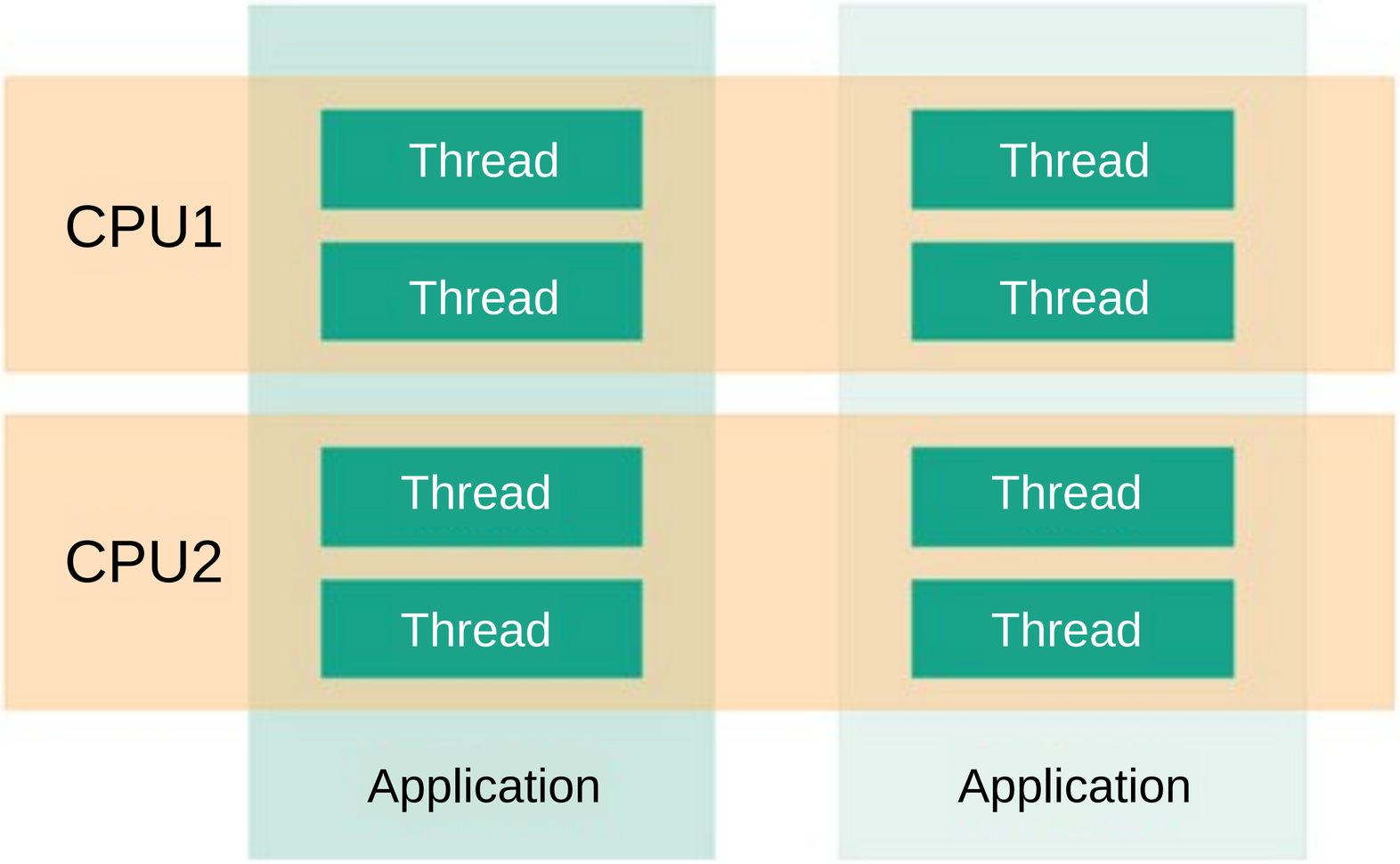


Concurrency



Parallel execution

Multithreading



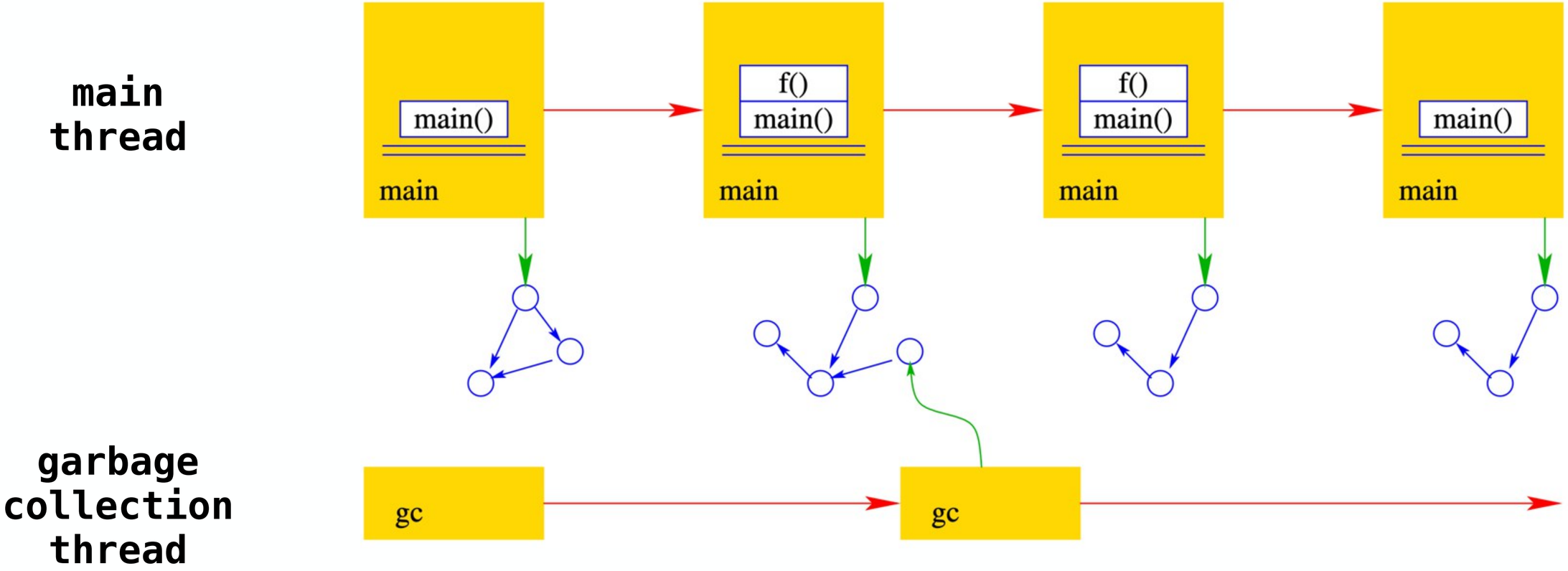
Motivation

- Shorter run duration for some long-running programs, thanks to parallel execution
- Better utilization of a single CPU
- Better utilization of multiple CPUs or CPU cores
- Better user experience with regards to responsiveness and fairness

Threads

- The execution of a program consists of **several threads** running in **parallel**
- A thread is a sequential execution path
- A program starts with the **`main`** thread, which executes the **`main()`** method of the program
- Another thread is used for **garbage collection** (started in parallel by the runtime system)
  - Eliminate objects that are no longer accessible in the meantime
  - Make the memory space occupied by them available for further program execution

Example: garbage collection



Threads

- Multiple threads are useful to
  - Monitor multiple input sources (e.g. mouse clicks and keyboard input)
  - Do something useful while a task is waiting or blocked
  - Use the computing power of several processors and cores
- New threads can be created and started by the program itself
- Java provides the class **`Thread`** and the interface **`Runnable`**

Graphical user interfaces

Variant 1

New threads are subclasses of the **Thread** class

```java
public class MyThread extends Thread {

    public void hello(String str) {
        System.out.println(str);
    }

    @Override
    public void run() {
        hello("I'm running ...");
    }

    public static void main(String[] args) {

        MyThread myThread = new MyThread();

        myThread.start();

        System.out.println("Thread has been started ...");
    }
}
```

Each subclass of **Thread** should implement this method

1. A new thread is initialized
2. The (parallel) execution of the object method **run()** for **myThread** is triggered
3. The own program execution is continued after the new thread is started

Variant 2

Implements the **Runnable** interface

```java
public class MyRunnable implements Runnable {

    public void hello(String str) {
        System.out.println(str);
    }

    @Override
    public void run() {
        hello("I'm running ...");
    }

    public static void main(String[] args) {

        Thread thread = new Thread(new MyRunnable());

        thread.start();

        System.out.println("Thread has been started ...");
    }
}
```

Each **runnable** must implement this method

1. A new thread is initialized
2. The (parallel) execution of the object method **run()** for the **Runnable** is triggered
3. The own program execution is continued after the new thread is started

Difference: the use of interface variant allows inheritance from another class

Variant 3 (with lambda expressions)

```
public class Playground {
    public static void main(String[] args) {
        Thread thread = new Thread(() -> System.out.println("I'm running ..."));
        thread.start();
        System.out.println("Thread has been started ...");
    }
}
```

The runnable is embedded in the lambda expression

Possible executions

```
Thread has been started ...
I'm running ...
```

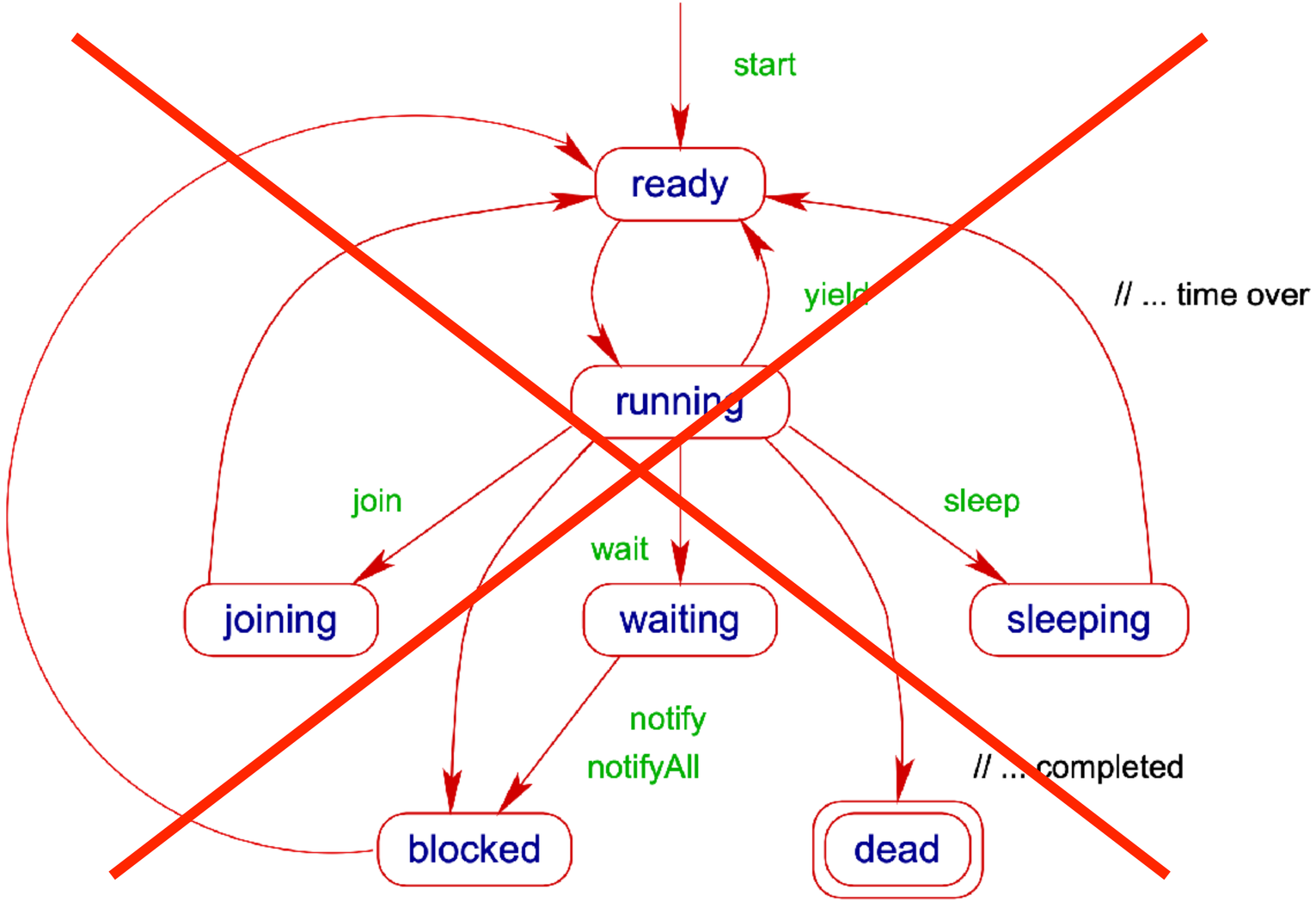
or

```
I'm running ...
Thread has been started ...
```

Scheduling

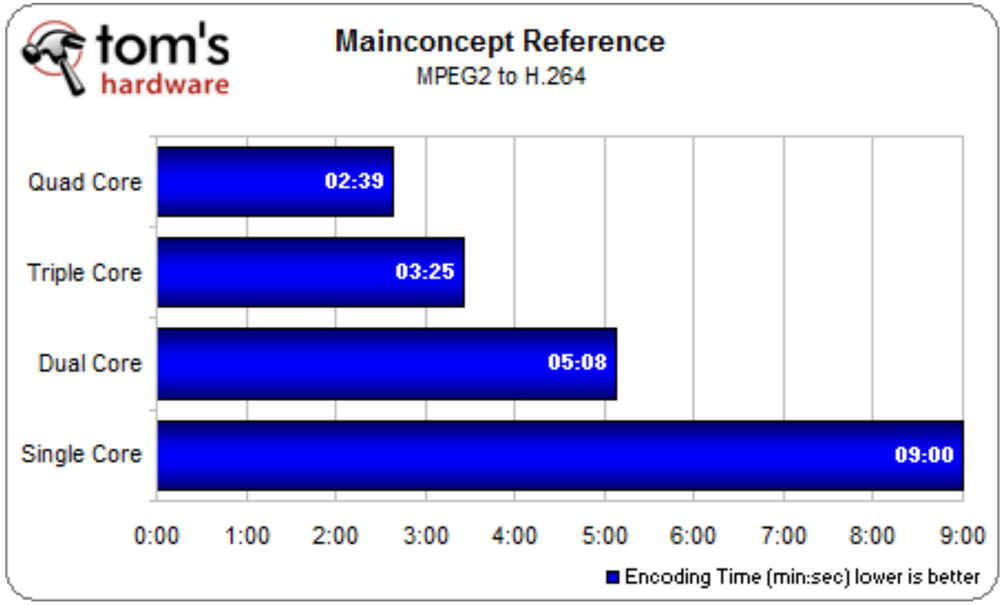
- A thread can only execute an operation if it has been allocated a processor (CPU) or core to execute it
- In general, there are more threads than CPUs
- The scheduler manages the available CPUs and allocates execution windows (time windows) to the threads
- For different program runs, this allocation can look different
- There are different policies that schedulers can follow (↑operating systems)
  - Time-slice method
  - Naive method
  - …

Java thread state model



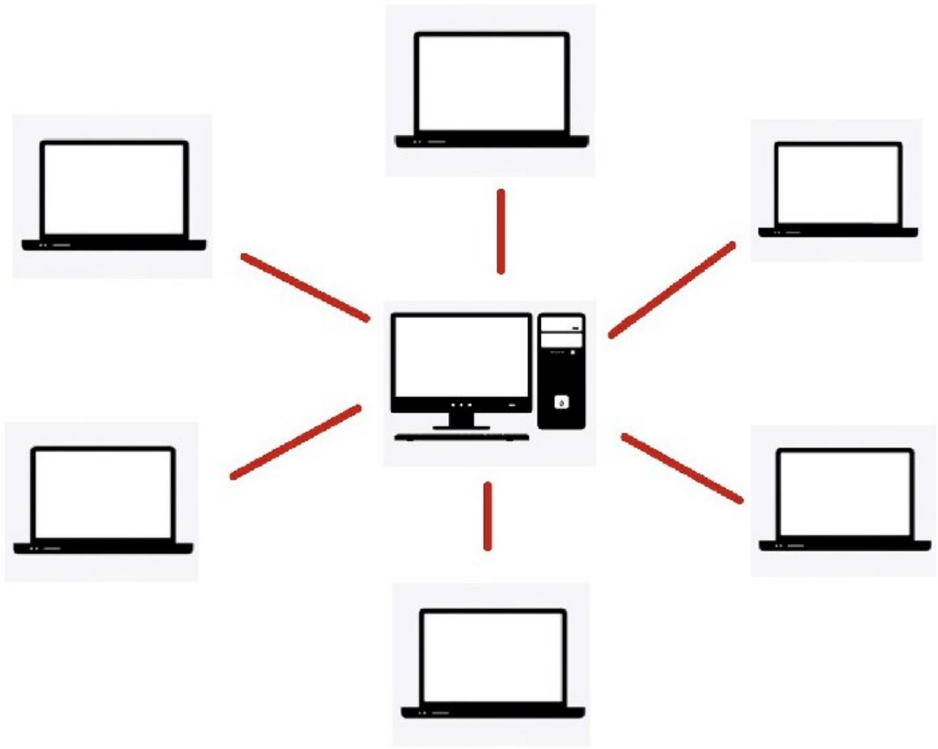
Concurrent programming: a foundational skill

- Modern software demands responsiveness and scalability
- Enables tasks to progress without blocking others
- Crucial for applications like web servers, real-time analytics, and data processing



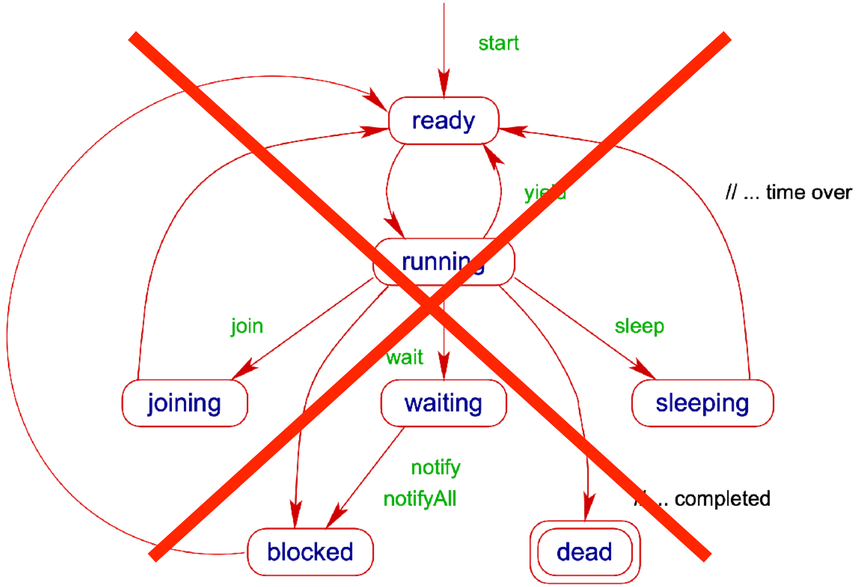
The role of distributed systems

- Communication between distributed components involves latency (network, database, input/output, APIs of external systems)
- Blocking operations lead to inefficiencies and poor user experience
- Concurrent programming optimizes resource usage and keeps systems responsive

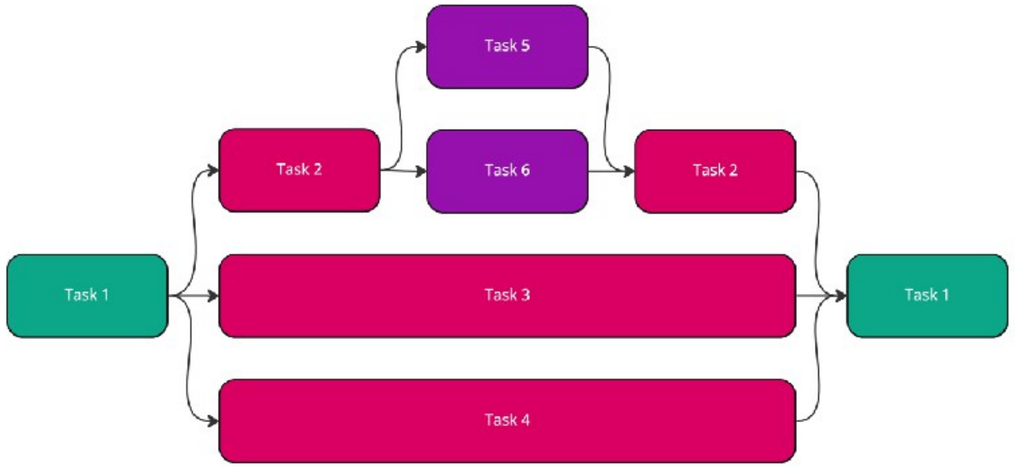


High-level approach to concurrency

- Abstracts away low-level thread management
- Focuses on “what” needs to happen, not “how”
- Builds scalable, maintainable systems for today’s interconnected world



Low-level thread management



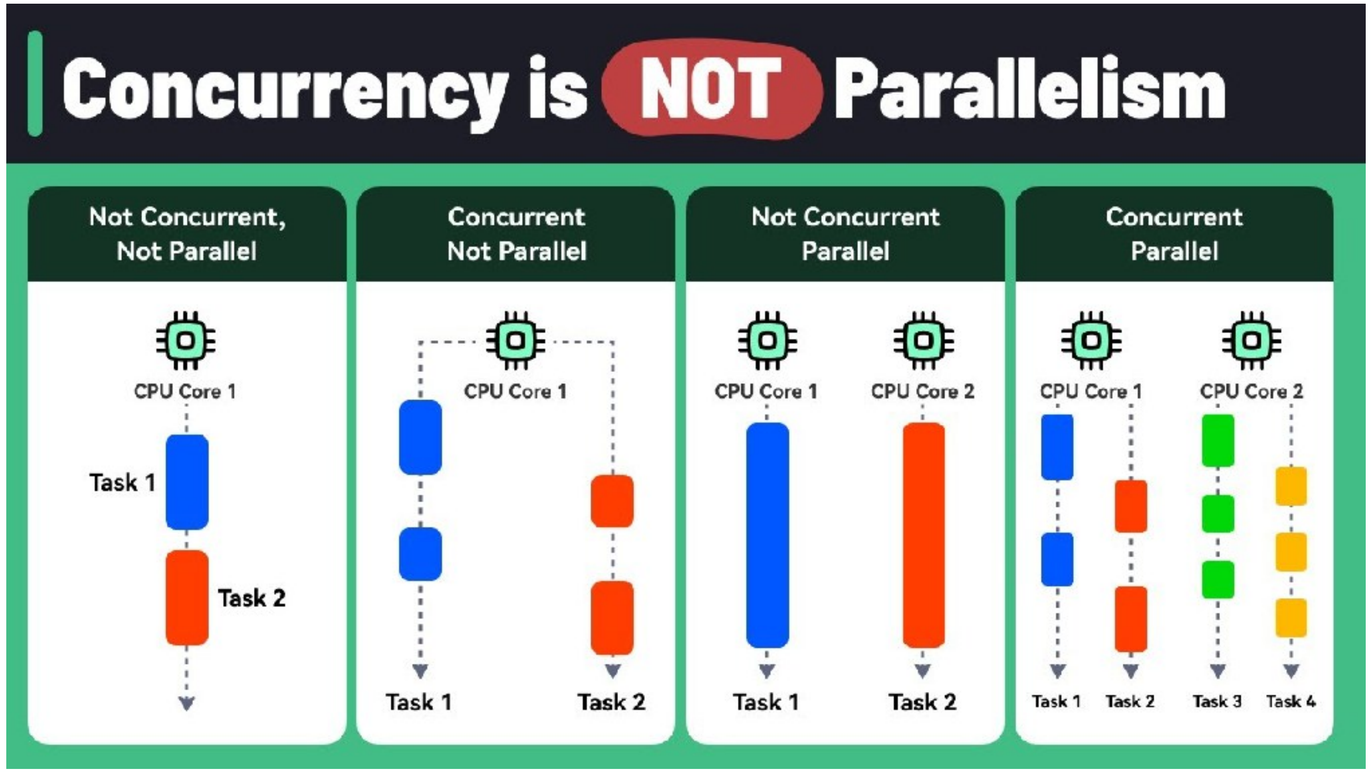
Structured **task based concurrency**

Outline

- Modern concurrency
- **Task-based concurrency**
- Completable futures
- Parallel streams
- Reactive programming

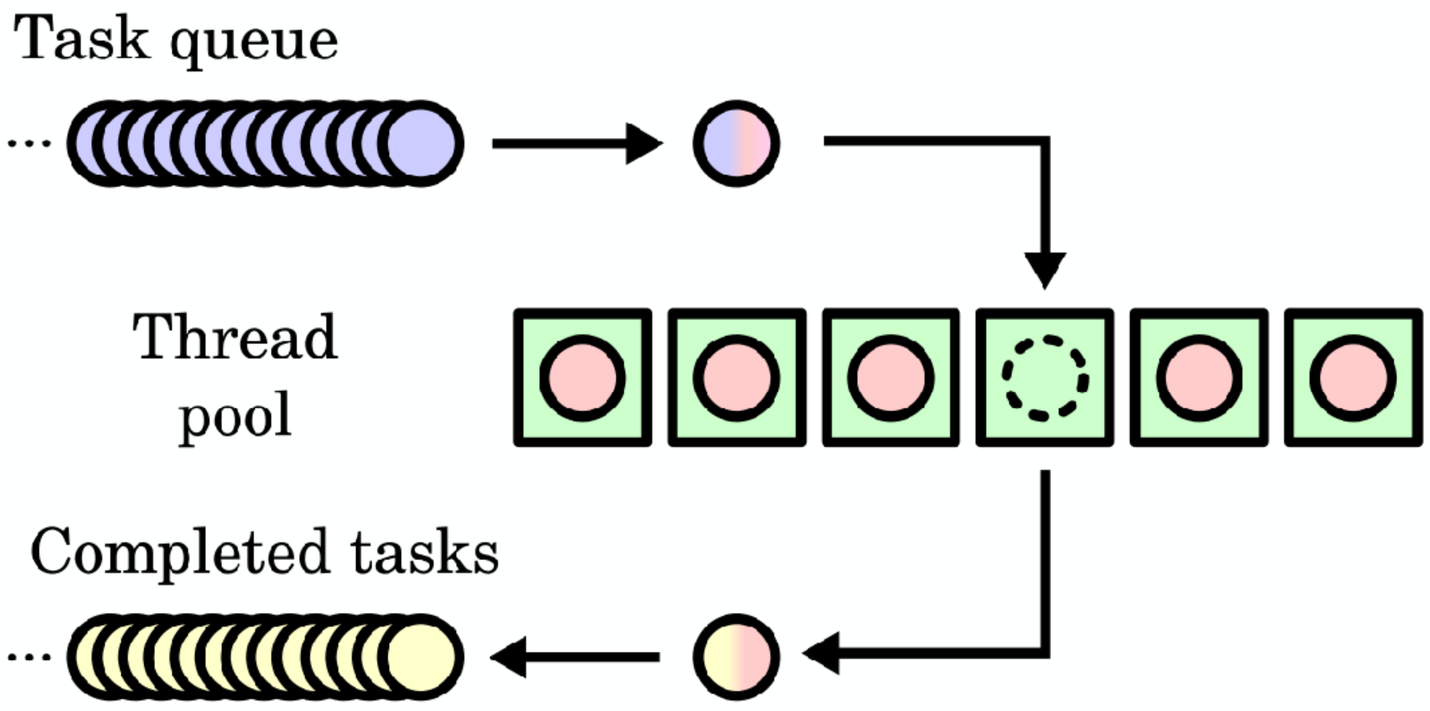
Parallelism vs. concurrency

- **Concurrency**: managing multiple tasks simultaneously (may or may not execute in parallel)
- **Parallelism**: tasks running at the same time on different cores



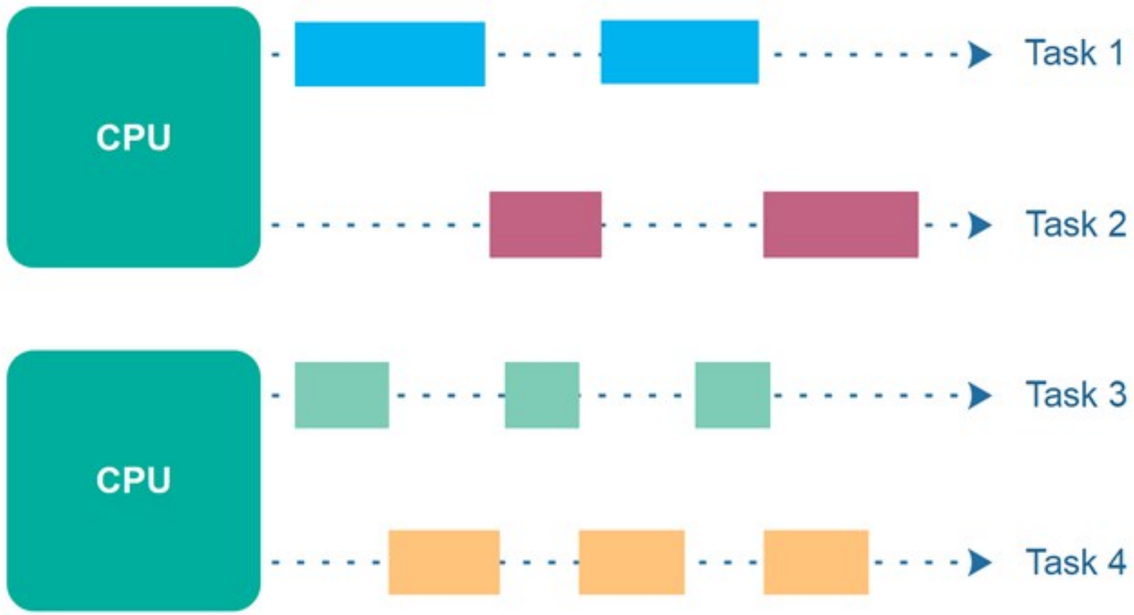
Modern concurrency abstractions

- Move away from low-level threads (complex, error-prone)
- Use **task-based concurrency** to simplify execution
- Thread pools manage reusable threads efficiently



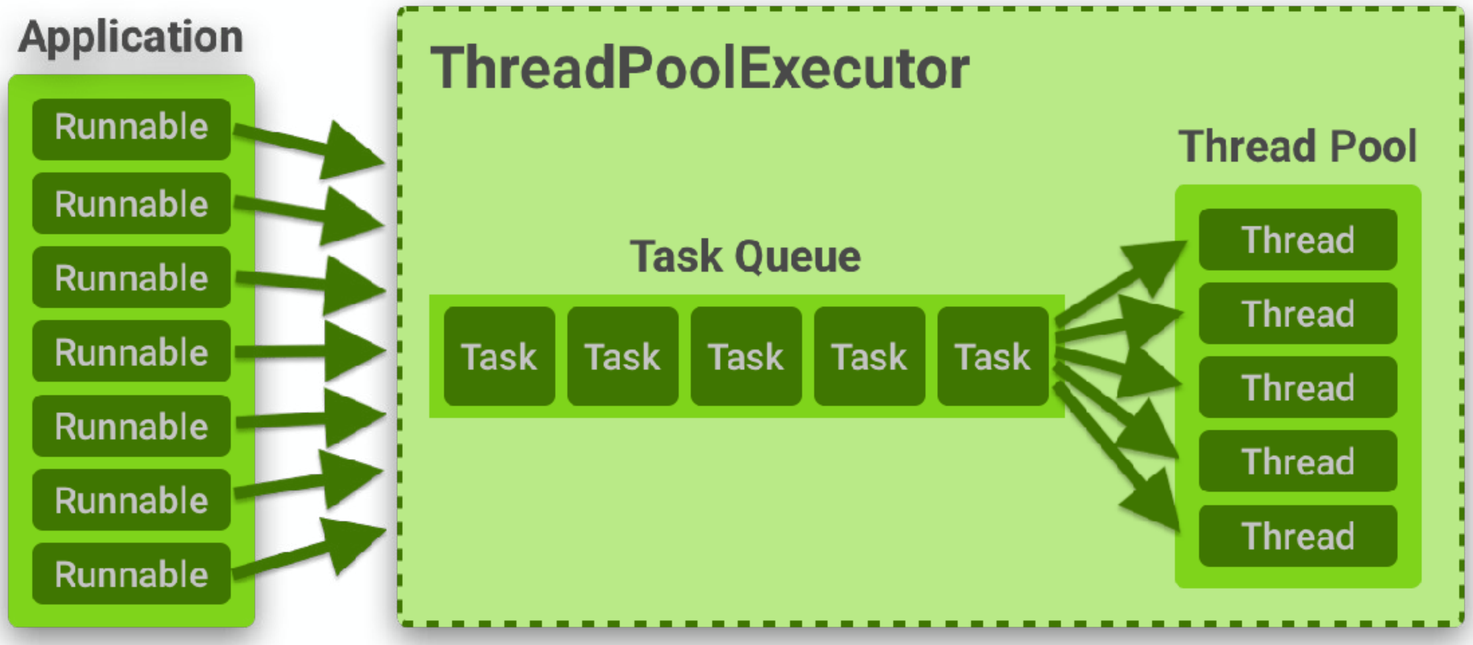
Task-based concurrency

- Tasks represent units of work
- Tasks are submitted to an executor for execution
- No need to manage threads manually



Introduction to thread pools

- Pool of reusable threads reduces overhead
- **`Executors`** provide predefined thread pool configurations like **`newFixedThreadPool(int nThreads)`** and **`newCachedThreadPool()`**
- Choose the right pool size for your workload



ExecutorService

```java
import java.util.concurrent.ExecutorService;
import java.util.concurrent.Executors;

public class Main {
    public static void main(String[] args) {
        ExecutorService executor = Executors.newFixedThreadPool(2);

        Runnable task1 = () -> System.out.println("Task 1 executed");
        Runnable task2 = () -> System.out.println("Task 2 executed");

        executor.submit(task1);
        executor.submit(task2);

        executor.shutdown();
    }
}
```

Order processing example

```java
import java.util.concurrent.ExecutorService;
import java.util.concurrent.Executors;

public class OrderProcessingExample {
    public static void main(String[] args) {
        ExecutorService executor = Executors.newFixedThreadPool(3);

        for (int orderId = 1; orderId <= 5; orderId++) {
            int finalOrderId = orderId; // Effectively final for lambda
            executor.submit(() -> {
                System.out.println("Processing Order #" + finalOrderId + " by " +
                                   Thread.currentThread().getName());
            });
        }

        executor.shutdown();
    }
}
```

User data fetch example

```java
import java.util.concurrent.Callable;
import java.util.concurrent.ExecutorService;
import java.util.concurrent.Executors;
import java.util.concurrent.Future;

public class UserDataFetchExample {
    public static void main(String[] args) throws Exception {
        ExecutorService executor = Executors.newFixedThreadPool(2);

        Callable<String> fetchUser = () -> {
            Thread.sleep(2000); // Simulate database delay
            return "User Data for User ID 42";
        };

        Future<String> result = executor.submit(fetchUser);

        System.out.println("Fetching user data...");
        System.out.println("Result: " + result.get()); // Blocks until result is available

        executor.shutdown();
    }
}
```

Notification service example

```java
import java.util.concurrent.Executors;
import java.util.concurrent.ScheduledExecutorService;
import java.util.concurrent.TimeUnit;

public class NotificationServiceExample {
    public static void main(String[] args) {
        ScheduledExecutorService scheduler = Executors.newScheduledThreadPool(2);

        Runnable sendNotification = () -> System.out.println("Notification sent at " +
                System.currentTimeMillis());

        scheduler.scheduleAtFixedRate(sendNotification, 0, 5, TimeUnit.SECONDS);

        // Schedule shutdown after 20 seconds for demonstration
        scheduler.schedule(scheduler::shutdown, 20, TimeUnit.SECONDS);
    }
}
```

Exercise

10 min

- Create a small program in your playground using **`ExecutorService`** to process a list of 100 integers in parallel and print their squares
- Use **`Executors.newFixedThreadPool(4)`** and submit tasks using a loop to square each integer
- **Hint:** You can use the following code structure to get started

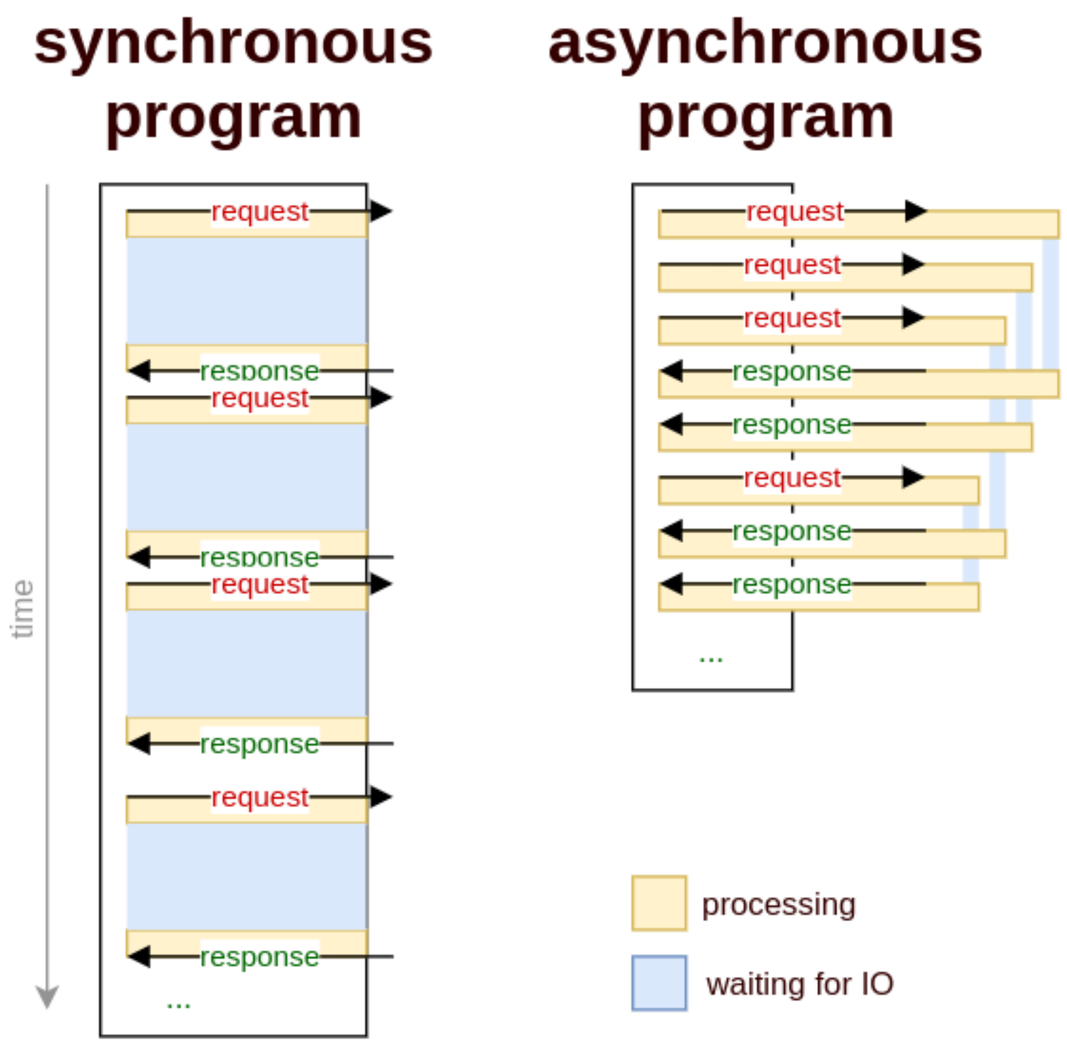
```java
import java.util.concurrent.*;
import java.util.stream.IntStream;

public class Main {
    public static void main(String[] args) {

        ExecutorService executor = null; // TODO: create an executor service
        var numbers = IntStream.rangeClosed(1, 100).boxed().toList();   // list of numbers from 1 to 100

        for (int number : numbers) {
            // TODO: submit the task to square the number to the executor service
        }

        executor.shutdown();
    }
}
```

Outline

- Modern concurrency
- Task-based concurrency
- **Completable futures**
- Parallel streams
- Reactive programming

Why asynchronous programming?

- Modern systems are increasingly event-driven and asynchronous
- Waiting for tasks to complete (e.g., network calls) would block execution and waste resources
- Asynchronous programming allows other work to continue while waiting for results



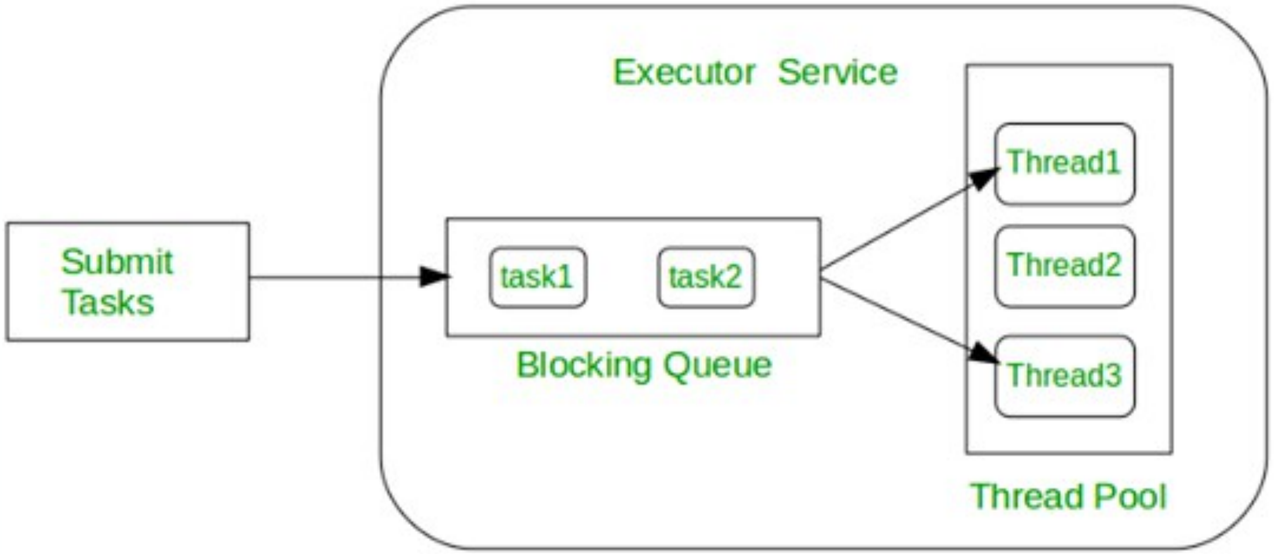
Futures: the basics

- **Promise**: a placeholder for a value that will be available in the future
- **Future**: represents a pending result of an asynchronous computation
- Used to retrieve results once the computation is done

```java
import java.util.concurrent.*;

public class FutureExample {
    public static void main(String[] args) throws ExecutionException, InterruptedException {
        ExecutorService executor = Executors.newSingleThreadExecutor();
        Future<Integer> future = executor.submit(() -> {
            Thread.sleep(2000); // Simulate long-running task
            return 42;
        });

        System.out.println("Task submitted...");
        System.out.println("Result: " + future.get()); // Blocks until the result is available
        executor.shutdown();
    }
}
```

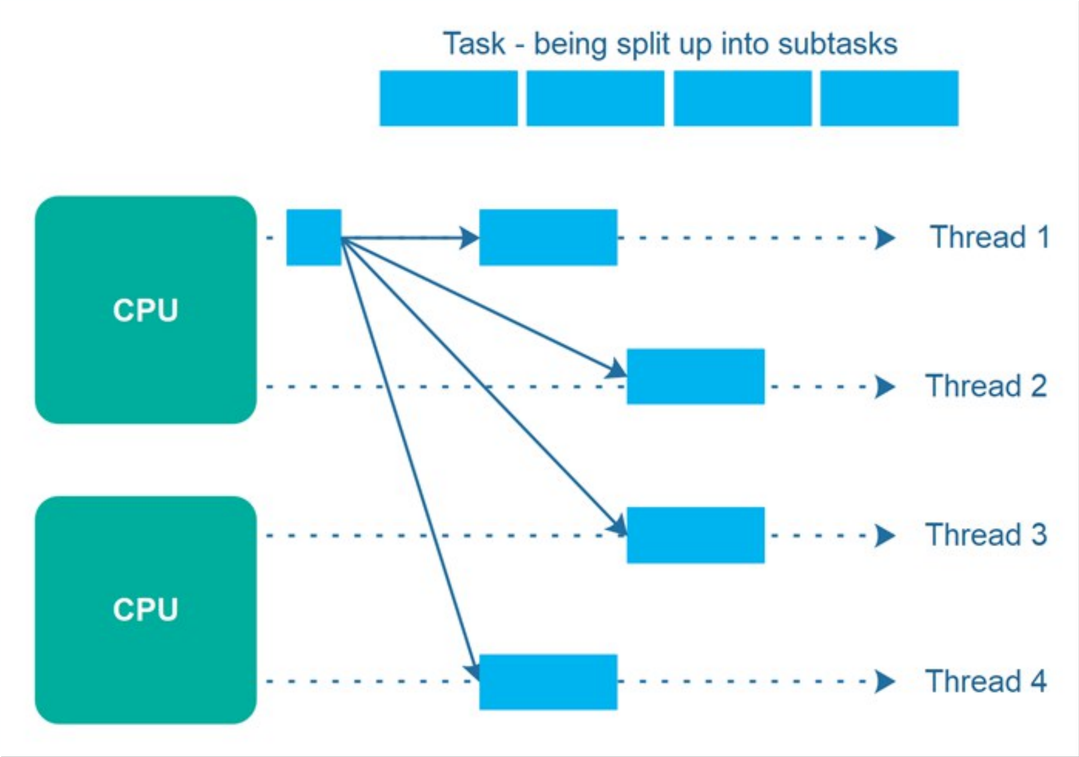
Problems with futures

- Futures are **blocking** when calling **`get()`**
- Cannot compose tasks directly (e.g., chaining or combining)
- Lack of built-in support for non-blocking callbacks



Completable future: a better future

- **`CompletableFuture`** was introduced in Java 8 as an extension of **`Future`**
- It supports chaining to define what happens after a task completes, combining results from multiple tasks, and handling exceptions gracefully



`CompletableFuture` example: basic usage

- Demonstrates asynchronous task execution, chaining transformations, and consuming the result

```
import java.util.concurrent.*;

public class CompletableFutureExample {
    public static void main(String[] args) {
        CompletableFuture.supplyAsync(() -> {
            System.out.println("Task running...");
            return "Hello";
        })
        .thenApply(result -> result + " World")
        .thenAccept(System.out::println);
    }
}
```

Composing tasks

- Use **`thenApply`** to transform results and **`thenCompose`** to chain dependent tasks (flat-mapping)

```
public class CompletableFutureExample {
    public static void main(String[] args) {
        CompletableFuture
            .supplyAsync(() -> "Task 1")
            .thenCompose(task1Result -> CompletableFuture
                    .supplyAsync(() -> task1Result + " + Task 2")
            )
            .thenAccept(System.out::println);
    }
}
```

Composing tasks

- Use **`thenCombine`** to merge results from two independent tasks

```
public class CompletableFutureExample {
    public static void main(String[] args) {
        CompletableFuture<Integer> task1 = CompletableFuture.supplyAsync(() -> 20);
        CompletableFuture<Integer> task2 = CompletableFuture.supplyAsync(() -> 22);

        task1.thenCombine(task2, Integer::sum)
                .thenAccept(result -> System.out.println("Sum: " + result));
    }
}
```

Exception handling with **CompletableFuture**

- Handle errors using **exceptionally()**

```
import java.util.concurrent.*;

public class CompletableFutureExample {
    public static void main(String[] args) {
        CompletableFuture.supplyAsync(() -> {
                    throw new RuntimeException("Error!");
                }).exceptionally(ex -> "Recovered from: " + ex.getMessage())
                .thenAccept(System.out::println);
    }
}
```

Exercise

10 min

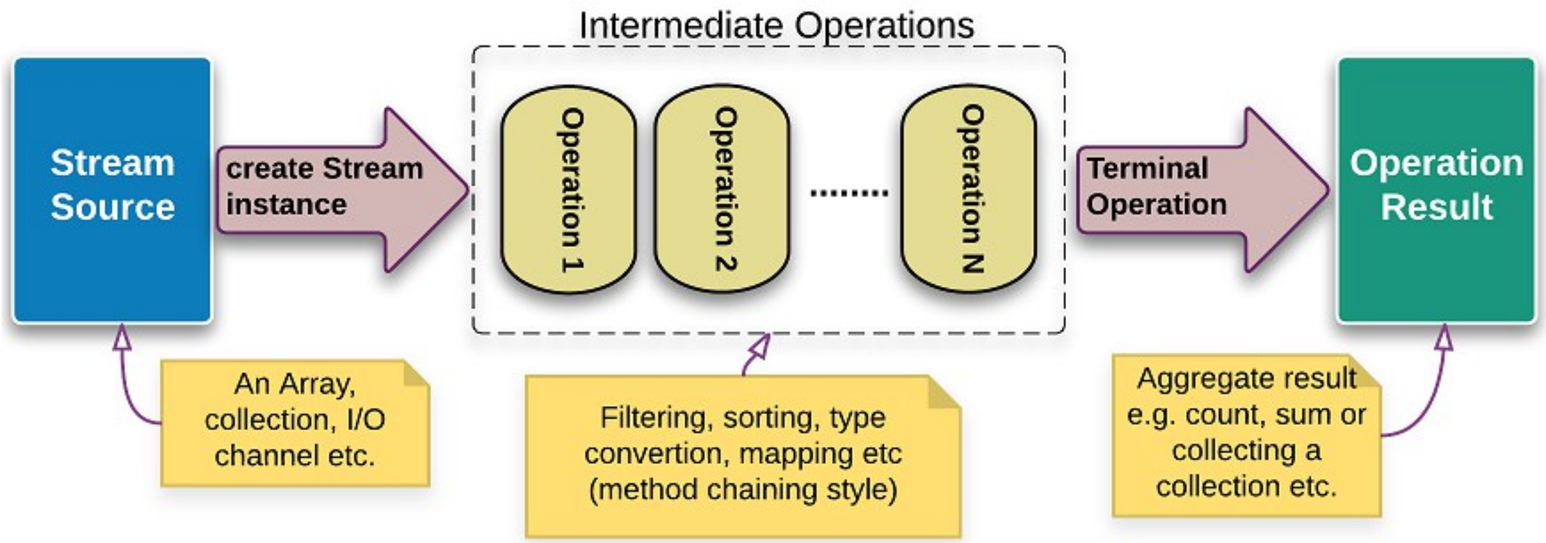
- Write a program that simulates fetching user data and order history asynchronously (simply return strings after using **`Thread.sleep(…)`**
- Use **`CompletableFuture.supplyAsync`** for each task
- Combine the results using **`thenCombine`** and display the final summary
- **Expected output**
  User: John Doe
  Orders: [Order1, Order2, Order3]

Outline

- Modern concurrency
- Task-based concurrency
- Completable futures
- **Parallel streams**
- Reactive programming

Review: streams

- Functional style operations on a discrete sequence of data elements
- Operate on the provided source and produce results rather than modifying the source
- A stream life cycle (also called pipeline) can be divided into three types of operation
  1. Create the stream instance
  2. Zero or more intermediate operations which transform a stream into another stream, such as filtering, sorting, element transformation (mapping)
  3. A terminal operation which produces a result, such as count, sum or a new collection



Functional programming in data processing

- Streams promote functional programming by focusing on “what” should be done rather than “how”
- Stream operations are declarative, making code concise and easier to read
- Functional programming simplifies data transformations and encourages immutability

Sequential vs. parallel streams

- Sequential streams process data one element at a time, in order
- Parallel streams split the data and process chunks concurrently using available processor cores.
- Parallelism is achieved without explicit thread management by the programmer

Code example

```
List<Integer> numbers = Arrays.asList(1, 2, 3, 4, 5);

// Sequential stream
numbers.stream()
        .map(n -> n * n)
        .forEach(System.out::println);

// Parallel stream
numbers.parallelStream()
        .map(n -> n * n)
        .forEach(System.out::println);
```

Parallel stream example

```
List<Integer> numbers = Arrays.asList(1, 2, 3, 4, 5);
numbers.parallelStream()
    .map(n -> {
        System.out.println("Processing " + n + " on thread " + Thread.currentThread().getName());
        return n * n;
    })
    .forEach(System.out::println);
```

Explanation: The **`parallelStream()`** method divides the list and processes elements concurrently across multiple threads

Use cases for parallel streams

- Parallel streams are ideal for CPU-bound tasks where data can be divided into independent chunks
- They work well for large data sets that require computationally intensive operations, such as mathematical transformations or aggregations

When **not** to use parallel streams

- Parallel streams are not suitable for small data sets where the overhead of splitting outweighs the benefits
- They are also unsuitable for IO-heavy operations, such as reading or writing files, due to thread contention
- Misuse can lead to performance degradation rather than improvement

Example – summing large numbers

```
import java.util.List;
import java.util.stream.LongStream;

public class ParallelStreams {
    public static void main(String[] args) {
        List<Long> largeNumbers = LongStream.range(1, 211_555_001).boxed().toList();
        long start = System.currentTimeMillis();
        long sum = largeNumbers.parallelStream().reduce(0L, Long::sum);
        long end = System.currentTimeMillis();
        System.out.println("Time taken: " + (end - start) + " ms");
        System.out.println("Sum: " + sum);
    }
}
```

Explanation: this example demonstrates a more efficient summation of a large range of numbers using parallel streams

Monitoring parallel streams

- Java allows monitoring of parallel streams using thread names or custom tools

```
import java.util.List;
import java.util.stream.LongStream;

public class ParallelStreams {
    public static void main(String[] args) {
        List<Long> numbers = LongStream.range(1, 500).boxed().toList();
        long start = System.currentTimeMillis();
        numbers.parallelStream()
                .map(n -> {
                    System.out.println("Thread: " + Thread.currentThread().getName() + " processing " + n);
                    return n * n;
                })
                .forEach(System.out::println);
        long end = System.currentTimeMillis();
        System.out.println("Time taken: " + (end - start) + " ms");
    }
}
```

Explanation: thread names in the output demonstrate how tasks are distributed across threads

W13E02 - Parallel Data Processing

Not started yet.

Start exercise

Easy

Due by tonight

10 min

3 pts

- **Problem statement:** Use parallel streams to process a large data set efficiently
  1. **Compute total sales revenue**: use parallel streams to calculate the total revenue
  2. **Find top 5 transactions**: use parallel streams to identify and sort the top 5 transactions in descending order
  3. **Measure sequential performance**: calculate the time (in ms) to sum the sales using regular streams
  4. **Measure parallel performance**: calculate the time (in ms) to sum the sales using **parallel** streams

Outline

- Modern concurrency
- Task-based concurrency
- Completable futures
- Parallel streams
- **Reactive programming**

Reactive manifesto

- **Responsive:** systems should respond in a timely manner
- **Message driven:** systems should use async message-passing between components to ensure loose coupling
- **Elastic:** systems should stay responsive under high load
- **Resilient:** systems should stay responsive when some components fail

- https://www.reactivemanifesto.org

Why reactive programming

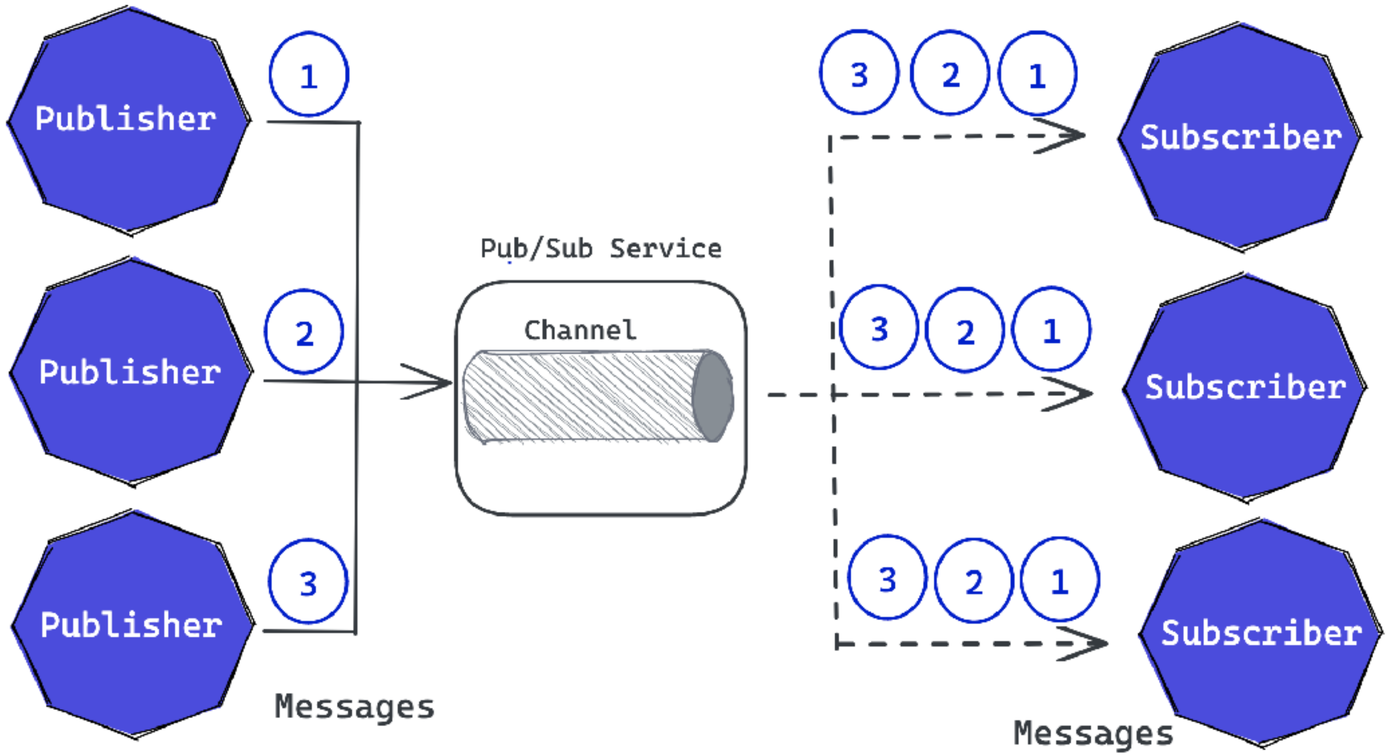
- Reactive programming is essential for handling large volumes of asynchronous data efficiently
- Real-world applications include microservices, real-time notifications, and data pipelines
- They require systems to remain responsive under high concurrency

Reactive programming

- Concerned with data streams and the propagation of change
- Change propagation algorithms
  - **Pull / polling**: the consumer is proactive, it regularly queries the observed source for values and reacts whenever a relevant value is available
  - **Push**: the consumer receives a self-contained value from the source whenever the value becomes available, no further information needs to be queried
  - **Push-pull**
    1. **Push:** the consumer receives a change notification, which is a short description of the change, e.g. "some value changed”, however, the notification does not contain all necessary information
    2. **Pull:** the consumer needs to query the source for more information (the specific value) after it receives the notification

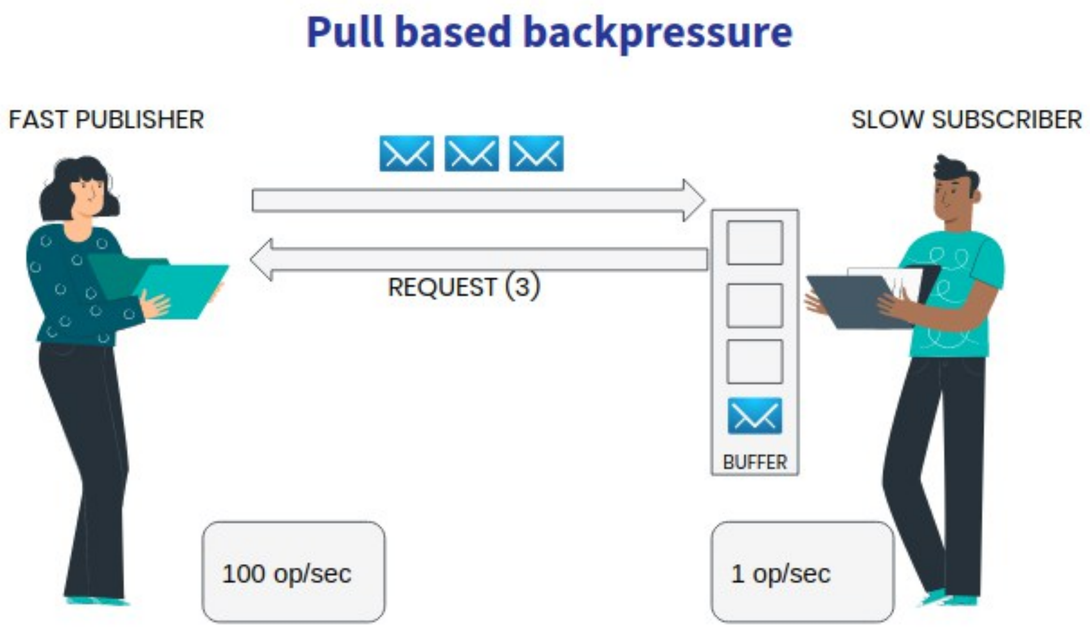
Publisher subscriber model

- The publisher-subscriber model decouples producers and consumers of data
- The publisher emits data, while the subscriber processes it
- This separation allows flexibility and scalability in reactive systems



What is back pressure?

- Back pressure manages the flow of data to avoid overwhelming subscribers
- It ensures that publishers send data at a rate that subscribers can handle, maintaining stability in high-throughput systems



Benefits of reactive programming in industry

- Reactive programming handles high concurrency and large-scale data streams efficiently
- It ensures resilience and responsiveness in real-world applications and aligns with modern architectures like microservices and serverless systems

Introduction to Java Flow API

- The Flow API, introduced in Java 9, implements the Reactive Streams specification
- It provides four main components
  - Publisher
  - Subscriber
  - Subscription
  - Processor

Basic components of Flow – publisher

- The publisher produces data and sends it to subscribers
- A publisher must implement the Flow Publisher interface

```
import java.util.concurrent.Flow;

public class SimplePublisher implements Flow.Publisher<String> {
    private final String[] data = {"Hello", "Reactive", "World"};

    @Override
    public void subscribe(Flow.Subscriber<? super String> subscriber) {
        // Subscription logic here
    }
}
```

Basic components of Flow – subscriber

- The subscriber consumes data emitted by the publisher
- A subscriber must implement the Flow Subscriber interface

```java
import java.util.concurrent.Flow;

public class SimpleSubscriber implements Flow.Subscriber<String> {
    @Override
    public void onSubscribe(Flow.Subscription subscription) {
        subscription.request(Long.MAX_VALUE);
    }

    @Override
    public void onNext(String item) {
        System.out.println("Received: " + item);
    }

    @Override
    public void onError(Throwable throwable) {}

    @Override
    public void onComplete() {
        System.out.println("All items processed.");
    }
}
```

Basic components of Flow – subscription

- The subscription connects a publisher to a subscriber and manages the flow of data
- It defines methods for requesting data and cancelling the subscription
- Code example

```java
import java.util.Arrays;
import java.util.concurrent.Flow;

public class Main {
    public static void main(String[] args) {

        var data = Arrays.asList("Hello", "Reactive", "World");
        SimpleSubscriber subscriber = new SimpleSubscriber();
        subscriber.onSubscribe(new Flow.Subscription() {
            private int index = 0;
            @Override
            public void request(long n) {
                while (index < data.size() && n > 0) {
                    subscriber.onNext(data.get(index++));
                    n--;
                }
                if (index == data.size()) {
                    subscriber.onComplete();
                }
            }
            @Override
            public void cancel() {}
        });
    }
}
```

Reactive publisher and subscriber

```
public class ReactiveExample {
    public static void main(String[] args) {
        var publisher = new SimplePublisher();
        publisher.subscribe(new SimpleSubscriber());
    }
}
```

Real-world application – data processing

- Reactive programming is ideal for event-driven systems like stock price updates, sensor data monitoring, or chat applications
- The publisher emits data events, and multiple subscribers can process them independently

When to use reactive programming

- Reactive programming is suitable for applications requiring high throughput, scalability, and responsiveness
- It is particularly useful in microservices, event-driven systems, and streaming data pipelines

W13E03 - Temperature Monitoring System

Not started yet.

Start exercise

Medium

Due by tonight

10 min

4 pts

- **Problem statement:** Simulate a temperature monitoring system using Java's Flow API
- System components
  - **Publisher**: emit random temperature readings in °C
  - **Subscriber**: log readings and warn for temperatures > 70°C
  - **Back pressure**: demonstrate controlled data consumption (10 readings at a time)
- **Scenario**: Monitor server room temperatures, alerting for unsafe levels
- Your tasks
  - Implement **TemperaturePublisher** to generate and manage data flow
  - Implement **TemperatureSubscriber** to log and warn for high temperatures
  - Create **ReactiveTemperatureMonitor** to test the system

Next steps

- **Tutor group** exercise
  - T13E01 - Divide and Conquer
- **Project work**
  - Implement the game
- Read the following articles
  - https://jenkov.com/tutorials/java-concurrency/concurrency-vs-parallelism.html
  - https://www.baeldung.com/java-completablefuture

→ Due by **Wednesday, January 28, 13:00**

Summary

- **Concurrency** ensures responsiveness; **parallelism** improves performance
- **Task-based concurrency** abstracts thread management for simplicity
- **Futures** represent results of **asynchronous computations** but are limited by blocking behavior, **CompletableFuture** allow for chaining, combining, and handling errors
- **Asynchronous programming** improves responsiveness and scalability in modern applications
- **Parallel streams** enable concurrency without thread management but require careful consideration of overhead and use cases
- **Reactive programming** provides a robust model for handling asynchronous data streams
- The **publisher-subscriber** model decouples data **producers** and **consumers**, while backpressure ensures stability

References

- https://jenkov.com/tutorials/java-concurrency/concurrency-vs-parallelism.html
- https://dev.to/adaumircosta/reactive-streams-and-the-flow-api-in-java-3eg6
- https://www.baeldung.com/java-when-to-use-parallel-stream
- https://www.baeldung.com/java-executor-service-tutorial
- https://jenkov.com/tutorials/java-util-concurrent/executorservice.html
- https://www.baeldung.com/java-completablefuture
- https://www.geeksforgeeks.org/completablefuture-in-java